

INŻYNIERIA OPROGRAMOWANIA

Dokument Architektury Systemu

System Zarządzania Reklamacjami Zwrotami – SZRZ

Architektura C4 · Schematy UML · Baza danych · REST API · Bezpieczeństwo

Mateusz Janicki

Rok akademicki 2025/2026

Wersja: 1.0 · Data: 2026-06-03

1. Wprowadzenie

1.1 Cel dokumentu

Niniejszy dokument opisuje architekturę techniczną Systemu Zarządzania Reklamacjami i Zwrotami (SZRZ). Zawiera: decyzje architektoniczne (ADR), model C4 na wszystkich poziomach, diagramy UML (komponentów, wdrożenia, sekwencji), pełny schemat bazy danych PostgreSQL, specyfikację endpointów REST API oraz opis architektury bezpieczeństwa.

1.2 Zasady architektoniczne

- Separation of Concerns – każda warstwa ma jedną odpowiedzialność.
- Fail Fast – walidacja na wejściu, szybkie zwracanie błędów zamiast silent failures.
- Defense in Depth – wiele warstw zabezpieczeń (sieć → auth → autoryzacja → dane).
- Idempotency – operacje API są idempotentne tam, gdzie to możliwe (PUT, retry powiadomień).
- Auditability – każda zmiana danych jest logowana z autorem i znacznikiem czasu (RODO).

2. Decyzje architektoniczne (ADR)

Architecture Decision Records dokumentują kluczowe wybory projektowe wraz z kontekstem i uzasadnieniem.

ADR-001 – Modular Monolith zamiast Mikrousług

Atrybut	Wartość
Status	Zaakceptowana
Kontekst	Projekt realizowany przez mały zespół (2–4 osoby). Mikrousługi wymagają złożonego zarządzania (service discovery, distributed tracing, sieć inter-service).
Decyzja	Zastosować architekturę modularnego monolitu opartego na Spring Boot z wyraźnymi granicami modułów (packages) odpowiadającymi domenom biznesowym.
Uzasadnienie	Upraszcza deployment, debugowanie i transakcje. Granice modułów umożliwiają ekstrakcję do mikroservisów w przyszłości bez przepisywania logiki biznesowej. Jedna baza danych eliminuje problemy distributed transactions.
Konsekwencje	Skalowanie całego systemu zamiast wybranych komponentów. Ryzyko naruszenia granic modułów wymaga code review.

ADR-002 – PostgreSQL jako główna baza danych

Atrybut	Wartość
Status	Zaakceptowana
Kontekst	Dane zgłoszeń mają silnie relacyjny charakter (klient → zamówienie → zgłoszenie → decyzja). Wymagana zgodność ACID dla operacji finansowych (zwroty środków). RODO wymaga precyzyjnego zarządzania danymi osobowymi.
Decyzja	PostgreSQL 16 jako jedyna relacyjna baza danych. Redis 7 jako cache i session store. Brak baz NoSQL w architekturze v1.
Uzasadnienie	PostgreSQL oferuje najlepsze wsparcie dla JSONB (elastyczne pola), Row Level Security (RBAC na poziomie BD), transakcje ACID i dojrzałe narzędzia migracji (Flyway).
Konsekwencje	Schemat musi być zaprojektowany z wyprzedzeniem. Migracje wymagają uwagi przy wdrożeniach zero-downtime.

ADR-003 – REST API (JSON/HTTP) zamiast GraphQL

Atrybut	Wartość
Status	Zaakceptowana
Kontekst	System ma zdefiniowany zestaw UC z przewidywalnymi wzorcami zapytań. Zespół ma doświadczenie w REST. Frontend jest prostą aplikacją SPA.
Decyzja	REST API z JSON, wersjonowanie przez prefiks /api/v1/, dokumentacja OpenAPI 3.0 generowana przez Springdoc.

Uzasadnienie	REST jest powszechnie zrozumiały, łatwiejszy do testowania (curl, Postman), nie wymaga dodatkowych bibliotek. GraphQL byłby nadmiarowy przy braku złożonych zapytań zagnieżdżonych.
Konsekwencje	Nadmiarowe endpointy przy nowych UC. Wersjonowanie wymaga zarządzania przy breaking changes.

ADR-004 – Asynchroniczne powiadomienia przez RabbitMQ

Atrybut	Wartość
Status	Zaakceptowana
Kontekst	Wysyłanie e-mail/SMS podczas obsługi HTTP requesta blokuje odpowiedź dla użytkownika. Bramka SMS może być niedostępna.
Decyzja	Każda zmiana statusu publikuje zdarzenie do kolejki RabbitMQ. Celery worker (Python) subskrybuje kolejkę i wysyła powiadomienia asynchronicznie.
Uzasadnienie	Odporność na awarie bramki (retry z exponential backoff). Brak blokowania głównego requesta. Możliwość podmiany bramki bez zmian w Backend API.
Konsekwencje	Złożoność infrastruktury wzrasta o RabbitMQ. Powiadomienia mogą być opóźnione (akceptowalne do 60 s).

ADR-005 – JWT (stateless) zamiast sesji serwerowych

Atrybut	Wartość
Status	Zaakceptowana
Kontekst	System musi skalować się poziomo (wiele instancji Backend API). Sesje serwerowe wymagają wspólnego session store lub sticky sessions.
Decyzja	JWT (HS256) jako access token (8h), refresh token (30 dni) przechowywany w httpOnly cookie. Role użytkownika zakodowane w JWT claim.
Uzasadnienie	Stateless – każda instancja Backend API weryfikuje token samodzielnie bez zapytania do Redis. Rola zakodowana w tokenie eliminuje DB lookup przy autoryzacji.
Konsekwencje	Odwołanie tokenów wymaga blacklisty w Redis (kompromis między stateless a bezpieczeństwem). Zmiana roli wymaga nowego tokena (akceptowalne).

3. Model C4 – architektura systemu

3.1 Poziom 1 – System Context (diagram kontekstowy)

Poziom 1 pokazuje SZRZ jako czarną skrzynkę w kontekście użytkowników i systemów zewnętrznych. Diagram kontekstowy C4 został szczegółowo opisany w dokumentacji Etapu I (model opisowy). Poniżej zestawienie relacji:

Element	Typ	Relacja z SZRZ
Klient	Użytkownik	Inicjuje zgłoszenia przez portal; sprawdza statusy; odbiera powiadomienia.
Pracownik obsługi	Użytkownik	Rejestruje i przetwarza zgłoszenia; podejmuje decyzje.
Kierownik	Użytkownik (uprz.)	Zatwierdza eskalacje; generuje raporty.
Administrator IT	Użytkownik (sys.)	Zarządza użytkownikami i konfiguracją.
System ERP	System zewnętrzny	Weryfikacja zamówień; przyjęcie zlecenia zwrotu środków (REST).
Platforma E-commerce	System zewnętrzny	Historię zamówień; aktualizacja statusów (webhooks).
WMS (Magazyn)	System zewnętrzny	Potwierdzenie odbioru towaru (REST).
Brama E-mail/SMS	System zewnętrzny	Wysyłka powiadomień (SMTP / SMS API).

3.2 Poziom 2 – Container Diagram (diagram kontenerów)

Poziom 2 rozбивa SZRZ na deployowalne jednostki (kontenery). Każdy kontener jest oddzielnym procesem z własną odpowiedzialnością.

Kontener	Technologia	Port/Protokół	Odpowiedzialność
Web Frontend	React 18 + TypeScript + Vite	HTTPS :443	Portal klienta + panel pracownika + dashboard. SPA komunikuje się wyłącznie z Backend API.
Backend API	Spring Boot 3 / Java 21	HTTP :8080 (internal)	Rdzeń logiki biznesowej: walidacja, autoryzacja JWT, reguły domenowe, orkestracja serwisów.
Notification Worker	Python 3.12 + Celery	RabbitMQ consumer	Asynchroniczne wysyłanie e-mail (SMTP) i SMS (Twilio). Retry z exponential backoff.
Report Engine	Python 3.12 + Pandas	HTTP :8090 (internal)	Generowanie raportów PDF za wybrany okres; agregacja danych z BD.
Integration Adapter	Node.js 20 + Express	HTTP :8081 (internal)	Proxy/adapter do ERP (REST), WMS (REST) i e-commerce (webhooks). Mapowanie formatów danych.
PostgreSQL 16	PostgreSQL + PgBouncer	TCP :5432 (internal)	Główna baza relacyjna. PgBouncer jako connection pool (max 100 conn).

Redis 7	Redis Cluster	TCP :6379 (internal)	Cache (wyniki zapytań, session data), kolejka zadań Celery, rate limiting counters.
RabbitMQ 3.x	RabbitMQ	AMQP :5672 (internal)	Broker wiadomości. Exchange: notifications. Queues: email_queue, sms_queue.
Nginx	nginx 1.24	HTTPS :443	Reverse proxy, TLS termination, load balancing między instancjami Backend API, serwowanie statycznych assetów.

3.3 Poziom 3 – Component Diagram (diagram komponentów Backend API)

Poziom 3 pokazuje wewnętrzną strukturę Backend API – komponenty w każdej warstwie i zależności między warstwami (strzałki skierowane w dół: kontrolery → serwisy → repozytoria).

Warstwa API (Controllers)

Komponent	Odpowiedzialność
ZgloszenieController	Obsługa endpointów /api/v1/zgloszenia. Przyjmuje żądania HTTP, waliduje DTO, deleguje do ZgloszenieService.
PracownikController	Endpointy dla panelu pracownika: aktualizacja statusu, rejestracja decyzji, inicjacja zwrotu.
KierownikController	Endpointy eskalacji i raportów dostępne dla roli KIEROWNIK i ADMINISTRATOR.
AdminController	Zarządzanie użytkownikami, konfiguracją terminów i regułami automatyzacji. Dostęp: ADMINISTRATOR.
AuthController	POST /api/v1/auth/login, /refresh, /logout. Generowanie i walidacja JWT.

Warstwa usług (Services)

Komponent	Odpowiedzialność
ZgloszenieService	Główna logika biznesowa: rejestracja zgłoszenia, walidacja zamówienia (ERP), generowanie numerów, zmiana statusu, kontrola uprawnień.
NotificationService	Publikuje zdarzenia do kolejki RabbitMQ przy każdej zmianie statusu. Buduje treść powiadomienia (szablony Thymeleaf).
EscalationService	Sprawdza terminy (harmonogram @Scheduled co 1h). Automatyczna eskalacja przy przekroczeniu lub sprzeciwie klienta.
TerminService	Oblicza termin rozpatrzenia na podstawie konfiguracji z tabeli konfiguracje_terminow. Alert przy progu.
ReportService	Deleguje generowanie raportów do Report Engine (HTTP call). Zwraca URL do pobrania PDF.
UserService	Zarządzanie cyklem życia kont użytkowników: tworzenie, edycja, blokowanie, zmiana hasła, audit log.

Warstwa repozytoriów (Repositories – Spring Data JPA)

Komponent	Odpowiedzialność
ZgloszenieRepository	CRUD + zapytania: findByKlientId, findByStatus, findByTerminBefore (dla alertów), findForReport.
KlientRepository	findByEmail (deduplication), findByNrZamowienia (proxy do ERP verify).
PracownikRepository	findByEmail (auth), findByRola (assignment), findAll (admin panel).
DecyzjaRepository	save (niemodyfikowalne przez @Immutable), findByZgloszenieId.
HistoriaStatusowRepository	Zapis każdej zmiany statusu. Używany do wyświetlania historii i raportów.

4. Schemat bazy danych (PostgreSQL 16)

Schemat składa się z 9 tabel. Wszystkie klucze główne to UUID (v4) generowane po stronie aplikacji. Wszystkie tabele zawierają kolumny systemowe: created_at TIMESTAMP DEFAULT NOW() i updated_at TIMESTAMP.

Tabela: klienci

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator klienta.
imie	VARCHAR(100)	NOT NULL	Imię klienta.
nazwisko	VARCHAR(100)	NOT NULL	Nazwisko klienta.
email	VARCHAR(255)	UNIQUE NOT NULL	Adres e-mail; używany do powiadomień i logowania.
telefon	VARCHAR(20)	NULL	Numer telefonu do SMS.
data_rejestracji	TIMESTAMP	DEFAULT NOW()	Data pierwszego zgłoszenia lub rejestracji.

Tabela: pracownicy

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator pracownika.
imie	VARCHAR(100)	NOT NULL	Imię pracownika.
nazwisko	VARCHAR(100)	NOT NULL	Nazwisko pracownika.
email	VARCHAR(255)	UNIQUE NOT NULL	Adres e-mail; używany do logowania.
password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt(12) hasła.
rola	ENUM	NOT NULL	PRACOWNIK_OBSLUGI KIEROWNIK ADMINISTRATOR.
aktywny	BOOLEAN	DEFAULT TRUE	FALSE = konto zablokowane (nie usuwa danych).

Tabela: zamowienia

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator zamówienia (lokalny).
nr_zamowienia	VARCHAR(50)	UNIQUE NOT NULL	Numer zamówienia z systemu ERP.
data_zakupu	TIMESTAMP	NOT NULL	Data złożenia zamówienia (z ERP).
kwota	DECIMAL(10,2)	NOT NULL	Wartość zamówienia (z ERP).

status_zamowienia	VARCHAR(50)	NULL	Status z ERP (DOSTARCZONE, W_REALIZACJI itd.).
klient_id	UUID	FK NOT NULL	Referencja do tabeli klienci.

Tabela: produkty

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator produktu.
nazwa	VARCHAR(255)	NOT NULL	Nazwa produktu.
kategoria	VARCHAR(100)	NULL	Kategoria (np. Smartfony, Laptopy).
nr_seryjny	VARCHAR(100)	NULL	Numer seryjny urządzenia (wymagany dla reklamacji gwarancyjnej).
zamowienie_id	UUID	FK NOT NULL	Referencja do tabeli zamowienia.

Tabela: zgloszenia

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator zgłoszenia.
nr_zgloszenia	VARCHAR(20)	UNIQUE NOT NULL	Czytelny numer: REC-2026-NNNNN lub ZWR-2026-NNNNN.
typ	ENUM	NOT NULL	REKLAMACJA ZWROT.
status	ENUM	NOT NULL	NOWE W_TRAKCIE OCZEKUJE_NA_TOWAR ROZPATRZONE ZAMKNIE ESKALOWANE.
kanal	ENUM	NOT NULL	ONLINE EMAIL TELEFON OSOBISCIE.
opis_problemu	TEXT	NOT NULL	Opis problemu podany przez klienta.
data_zgloszenia	TIMESTAMP	NOT NULL DEFAULT NOW()	Data i czas złożenia zgłoszenia.
termin_rozpatrzenia	DATE	NOT NULL	Obliczany na podstawie konfiguracji_terminow.
klient_id	UUID	FK NOT NULL	Referencja do klienci.
pracownik_id	UUID	FK NULL	Przypisany pracownik (nullable przy rejestracji).
zamowienie_id	UUID	FK NOT NULL	Referencja do zamowienia.

Tabela: decyzje

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator decyzji.

typ	ENUM	NOT NULL	NAPRAWA WYMIANA ZWROT_GOTOWKI OBNIZENIE_CENY ODRZUCENIE ESKALACJA.
uzasadnienie	TEXT	NOT NULL	Opis uzasadnienia decyzji (obowiązkowy).
kwota_zwrotu	DECIMAL(10,2)	NULL	Kwota zwrotu (dla ZWROT_GOTOWKI i OBNIZENIE_CENY).
data_decyzji	TIMESTAMP	NOT NULL DEFAULT NOW()	Data i czas zatwierdzenia decyzji.
zgloszenie_id	UUID	FK UNIQUE NOT NULL	Relacja 1:1 z zgloszenia.
pracownik_id	UUID	FK NOT NULL	Pracownik/kierownik który zatwierdził decyzję.

Tabela: historia_statusow

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator wpisu historii.
stary_status	VARCHAR(30)	NOT NULL	Status przed zmianą.
nowy_status	VARCHAR(30)	NOT NULL	Status po zmianie.
data_zmiany	TIMESTAMP	NOT NULL DEFAULT NOW()	Znacznik czasu zmiany.
komentarz	TEXT	NULL	Opcjonalny komentarz pracownika.
zgloszenie_id	UUID	FK NOT NULL	Referencja do zgloszenia.
pracownik_id	UUID	FK NULL	NULL jeśli zmiana automatyczna (system).

Tabela: powiadomienia

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator powiadomienia.
typ	ENUM	NOT NULL	EMAIL SMS.
tresc	TEXT	NOT NULL	Treść powiadomienia (HTML dla email, plain text dla SMS).
data_poslania	TIMESTAMP	NULL	Null do momentu faktycznego wysłania.
dostarczone	BOOLEAN	DEFAULT FALSE	True po potwierdzeniu dostarczenia.
zgloszenie_id	UUID	FK NOT NULL	Referencja do zgloszenia.
klient_id	UUID	FK NOT NULL	Referencja do klienci.

Tabela: konfiguracje_terminow

Kolumna	Typ danych	Ograniczenia	Opis
id	UUID	PK NOT NULL	Unikalny identyfikator konfiguracji.
typ_zgloszenia	ENUM	UNIQUE NOT NULL	REKLAMACJA ZWROT.
termin_dni	INT	NOT NULL	Liczba dni na rozpatrzenie (14 lub 30).
prog_alertu_dni	INT	NOT NULL DEFAULT 2	Liczba dni przed terminem do wysłania alertu.
zmienione_przez	UUID	FK NOT NULL	Referencja do pracownicy (administrator).
data_zmiany	TIMESTAMP	NOT NULL DEFAULT NOW()	Data zmiany konfiguracji (audit log).

5. Specyfikacja REST API

Wszystkie endpointy są poprzedzone prefiksem `/api/v1/`. Autentykacja przez nagłówek `Authorization: Bearer <JWT>`. Format: JSON (Content-Type: `application/json`). Szczegółowa specyfikacja w formacie OpenAPI 3.0 jest generowana automatycznie przez Springdoc pod `/api/docs`.

5.1 Zasoby i endpointy

Met.	Endpoint	Status	Rola	Opis
<code>/zgl osz enia</code>	201	Klient, Prac.	Rejestracja zgłoszenia. Body: { nrZamowieni a, typ, opis, kanal }. Weryfikacja ERP inline.	
<code>/zgl osz enia</code>	200	Prac., Kier.	Lista zgłoszeń z filtrowaniem (status, typ, data, klient). Paginacja: page, size.	
<code>/zgl osz enia /{id}</code>	200	Klient*, Prac.	Szczegóły + historia statusów. *Klient widzi tylko własne.	
<code>/zgl osz enia /{id} /stat us</code>	200	Pracow nik	Zmiana statusu + wpis w historii + async powiadomienie.	
<code>/zgl osz enia /{id} /dec yzja</code>	201	Pracow nik	Zapis niemodyfikowalnej decyzji końcowej.	
<code>/zgl osz enia /{id} /esk alac ja</code>	200	Prac., System	Eskalacja → status ESKALOWANE + alert do kierownika.	
<code>/zgl osz enia /{id}</code>	200	Klient*, Prac.	Pełna historia zmian statusów.	

/historia			
/zwroty/{id}/etykieta	200	Pracownik	Generuj etykietę PDF + zmiana statusu.
/zwroty/{id}/odbior	200	System (WMS)	Potwierdzenie odbioru towaru przez magazyn (webhook).
/zamowienia/{nr}	200	Backend	Proxy do ERP: weryfikacja zamówienia. Zwraca { klient, produkty, kwota }.
/raporty	200	Kierownik	Raport z filtrami { dateFrom, dateTo, typ, status }. Query param: format=pdf.
/uzytkownicy	201	Admin	Utwórz konto z przypisaniem roli.
/uzytkownicy/{id}	200	Admin	Edycja konta (rola, aktywny).
/auth/login	200	*	{ email, password } → { accessToken, refreshToken }.
/auth/refresh	200	*	{ refreshToken } → nowy accessToken.

5.2 Standardowe odpowiedzi błędów

Kod HTTP	Kod błędu	Opis
400 Bad Request	VALIDATION_ERROR	Nieprawidłowe dane wejściowe. Body: { errors: [{ field, message }] }.

401 Unauthorized	INVALID_TOKEN	JWT wygasły lub nieprawidłowy.
403 Forbidden	ACCESS_DENIED	Brak uprawnień dla roli użytkownika.
404 Not Found	RESOURCE_NOT_FOUND	Zasób nie istnieje lub klient nie ma do niego dostępu.
409 Conflict	DUPLICATE_ZGLOSZENIE	Zgłoszenie dla tego zamówienia już istnieje (status aktywny).
422 Unprocessable	BUSINESS_RULE_VIOLATION	Naruszenie reguły biznesowej (np. zmiana statusu na niedozwolony).
503 Service Unavailable	ERP_UNAVAILABLE	System ERP niedostępny; zgłoszenie buforowane.

6. Architektura bezpieczeństwa

6.1 Warstwy zabezpieczeń (Defense in Depth)

Warstwa 1 – Sieć

- HTTPS/TLS 1.3 obowiązkowy dla całej komunikacji. HSTS z max-age=31536000.
- Certyfikat SSL automatycznie odnawiany przez Let's Encrypt (certbot).
- Firewall: tylko port 443 (HTTPS) i 22 (SSH z IP whitelist) eksponowane publicznie.
- Rate limiting: max 100 req/min per IP (nginx), max 10 req/min dla /auth/* (brute force).
- CDN z ochroną przed DDoS (Cloudflare lub AWS CloudFront).

Warstwa 2 – Autentykacja

- JWT (HS256); access token 8h, refresh token 30 dni (httpOnly, Secure, SameSite=Strict cookie).
- Blokada konta po 5 nieudanych próbach logowania na 15 minut (counter w Redis).
- Hasła hashowane algorytmem bcrypt(cost=12).
- Polityka haseł: min. 12 znaków, wymagane: wielka litera, cyfra, znak specjalny.
- Obowiązkowa zmiana hasła co 90 dni dla pracowników.

Warstwa 3 – Autoryzacja (RBAC)

Zasób / akcja	Klient	Pracownik	Kierownik	Administrator
Rejestracja zgłoszenia	✓ (własne)	✓ (w imieniu klienta)	✓	✓
Zmiana statusu	✗	✓	✓	✓
Decyzja końcowa	✗	✓	✓ (nadrzędna)	✓
Eskalacja	✗	✓	✓	✓
Raporty analityczne	✗	✗	✓	✓
Zarządzanie użytkownikami	✗	✗	✗	✓
Konfiguracja terminów	✗	✗	✗	✓
Podgląd cudzych zgłoszeń	✗	✓	✓	✓

Warstwa 4 – Ochrona danych (RODO)

- Szyfrowanie danych osobowych (imię, nazwisko, email, telefon) w spoczynku: AES-256-GCM.
- Logi dostępu do danych osobowych przechowywane przez 2 lata (audit trail).
- Retencja danych zgłoszeń: 5 lat od zamknięcia. Po tym czasie: anonimizacja lub usunięcie na żądanie.
- Prawo do zapomnienia: endpoint DELETE /klienci/{id}/dane (pseudonimizacja, nie usunięcie FK).
- Połączenia do PostgreSQL z SSL mode: require (szyfrowanie transportu wewnętrznego).

7. Stos technologiczny

Wybór technologii podyktowany: doświadczeniem zespołu, dojrzałością ekosystemu, otwartą licencją (OSS) oraz dopasowaniem do wymagań NFR.

Warstwa	Technologia / Wersja	Licencja	Uzasadnienie wyboru
Frontend	React 18 + TypeScript 5	MIT	Największy ekosystem, TypeScript eliminuje błędy typów, Server Components (React 18) dla SEO.
Frontend	Tailwind CSS 3	MIT	Utility-first CSS; szybki prototyping; brak konfliktu stylów.
Backend	Spring Boot 3.x (Java 21)	Apache 2.0	De facto standard w enterprise Java. Virtual Threads (Project Loom) dla wysokiej przepustowości.
Backend	Spring Security 6 + JWT	Apache 2.0	Dojrzała obsługa RBAC, integracja z Spring Boot, @PreAuthorize na metodach.
Backend	JPA/Hibernate 6	LGPL	ORM dla PostgreSQL. Flyway do migracji schematu BD.
Baza danych	PostgreSQL 16	PostgreSQL Lic.	ACID, JSONB, Row Level Security, silna obsługa UUID, dojrzałe narzędzia.
Cache/Queue	Redis 7	BSD-3	Wielofunkcyjny: cache, rate limiting, sesje, kolejka Celery.
Messaging	RabbitMQ 3.x	MPL-2.0	Dojrzały AMQP broker; dead-letter queues; management UI.
Notifications	Celery + Python 3.12	BSD	Asynchroniczne workers, retry/backoff, monitoring przez Flower.
Reports	Python 3.12 + Pandas + ReportLab	BSD/LGPL	Pandas do agregacji danych; ReportLab do generowania PDF.
API Docs	OpenAPI 3.0 (Springdoc)	Apache 2.0	Auto-generowana spec z adnotacji Spring; Swagger UI /api/docs.
CI/CD	GitHub Actions	MIT	Pipelines: test → build → docker push → deploy. Secrets w GitHub Secrets.
Containers	Docker + Docker Compose	Apache 2.0	Spójne środowisko dev/prod. Compose dla dev, K8s opcjonalnie w prod.
Monitoring	Prometheus + Grafana	Apache 2.0	Metryki JVM, BD, kolejek. Alerty przy przekroczeniu SLA.
Proxy	Nginx 1.24	BSD	Reverse proxy, TLS termination, load balancing, cache statycznych assetów.

8. Podsumowanie i kolejne etapy

Zrealizowane w tym etapie

- 5 Architecture Decision Records z kontekstem, decyzją i uzasadnieniem.
- Model C4: poziom 1 (kontekst), poziom 2 (9 kontenerów), poziom 3 (18 komponentów Backend).
- Diagram wdrożenia: 4 tiery (klient, aplikacja, dane, zewnętrzne systemy).
- Schemat bazy danych: 9 tabel z pełnymi definicjami kolumn, typami, ograniczeniami.
- Specyfikacja REST API: 15 endpointów z metodami, statusami i zakresami dostępu.
- Architektura bezpieczeństwa: 4 warstwy (sieć, autentykacja, RBAC, dane/RODO).
- Stos technologiczny: 15 technologii z uzasadnieniem wyboru i informacją o licencji.

Kolejne etapy

- Implementacja – setup projektu Spring Boot, inicjalizacja bazy danych (Flyway), szkielet API.
- Testy – implementacja testów jednostkowych (JUnit 5) i integracyjnych (Testcontainers + PostgreSQL).
- Makiety UI – szczegółowe wireframes portalu klienta i panelu pracownika.
- Plan iteracji – harmonogram sprintów, backlog zadań, definicja 'Done'.
- Deployment – konfiguracja Docker Compose dla środowiska deweloperskiego.